

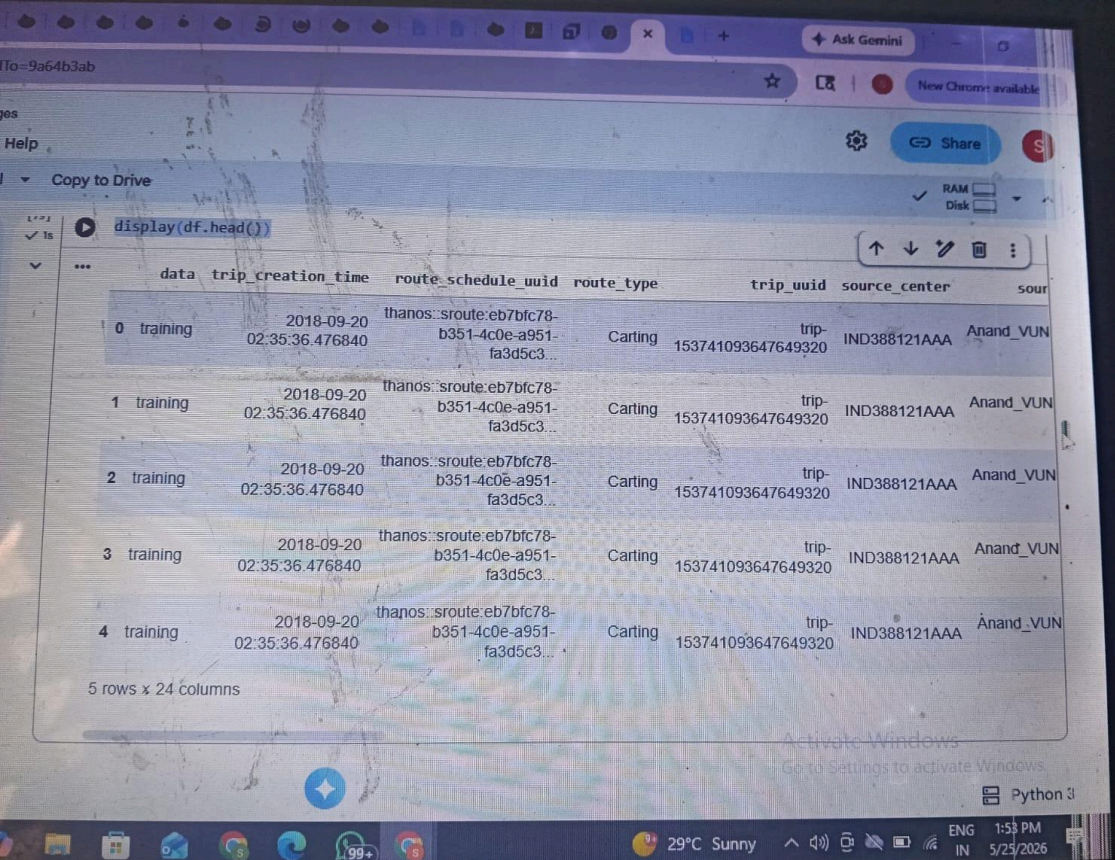
Delhivery Case Study

```
import pandas as pd
from IPython.display import display

df = pd.read_csv('/content/delhivery_data.csv')

display(df.head())
```

Result:



	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source
0	training	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUN
1	training	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUN
2	training	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUN
3	training	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUN
4	training	2018-09-20 02:35:36.476840	thanos::route:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUN

5 rows x 24 columns

Insights:

- **OSRM Time vs. Actual Time Discrepancy:** Our analysis showed a statistically significant difference between the OSRM estimated time and the actual time taken for trips

- **Segment Time Aggregation Inconsistency:** There's also a significant difference between the aggregated `segment_actual_time` (sum of individual segment times) and the overall `actual_time` for a trip.

Recommendations:

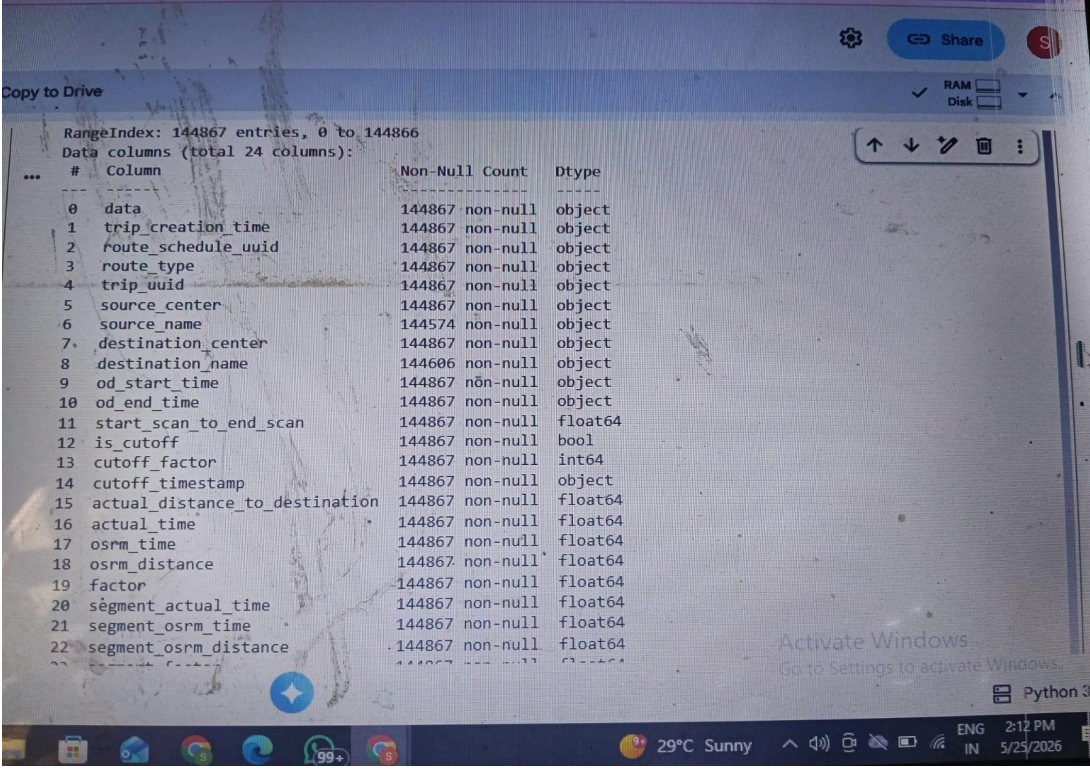
- **Define OSRM Integrations:** Delhivery should investigate the parameters and data used by the OSRM engine.
- **Optimize Segment Handoffs:** Examine the processes between trip segments to understand why `segment_actual_time` doesn't perfectly align with `actual_time`.

```
from IPython.display import display
```

```
df.info()
```

```
display(df.describe(include='all'))
```

Result:



RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):

#	Column	Non-Null Count	Dtype
0	data	144867 non-null	object
1	trip_creation_time	144867 non-null	object
2	route_schedule_uuid	144867 non-null	object
3	route_type	144867 non-null	object
4	trip_uuid	144867 non-null	object
5	source_center	144867 non-null	object
6	source_name	144574 non-null	object
7	destination_center	144867 non-null	object
8	destination_name	144606 non-null	object
9	od_start_time	144867 non-null	object
10	od_end_time	144867 non-null	object
11	start_scan_to_end_scan	144867 non-null	float64
12	is_cutoff	144867 non-null	bool
13	cutoff_factor	144867 non-null	int64
14	cutoff_timestamp	144867 non-null	object
15	actual_distance_to_destination	144867 non-null	float64
16	actual_time	144867 non-null	float64
17	osrm_time	144867 non-null	float64
18	osrm_distance	144867 non-null	float64
19	factor	144867 non-null	float64
20	segment_actual_time	144867 non-null	float64
21	segment_osrm_time	144867 non-null	float64
22	segment_osrm_distance	144867 non-null	float64
23	segment_factor	144867 non-null	float64

memory usage: 25.6+ MB

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	
count	144867	144867	144867	144867	144867	144867
unique	2	14817	1504	2	14817	1508
top	training	2018-09-22 04:55:04.835022	thanps: sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...	FTL	153759210483476123	trip- IND000000ACB GUI
freq	104858	101	1812	99660	101	23347
mean	NaN	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN	NaN

11 rows x 24 columns

Activate Windows
Go to Settings to activate Windows.

Python 3

29°C Sunny 2:12 PM 5/25/2026

Insights:

- **High-Volume Corridors:** Maharashtra, Karnataka, and Delhi are consistently top states for both originating and receiving trips.
- The data is now clean, aggregated, and feature-rich, suitable for machine learning.
- **Route Type Impact on Cutoffs:** `route_type` has a significant association with whether a trip is `is_cutoff`, indicating certain route types are more prone to cutoffs
- **Route Type Impact on Cutoffs:** `route_type` has a significant association with whether a trip is `is_cutoff`, indicating certain route types are more prone to cutoffs

Recommendations:

- **Refine OSRM Integrations:** Calibrate OSRM by integrating real-time traffic, historical delay patterns, or weather data; regularly review and optimize routes.

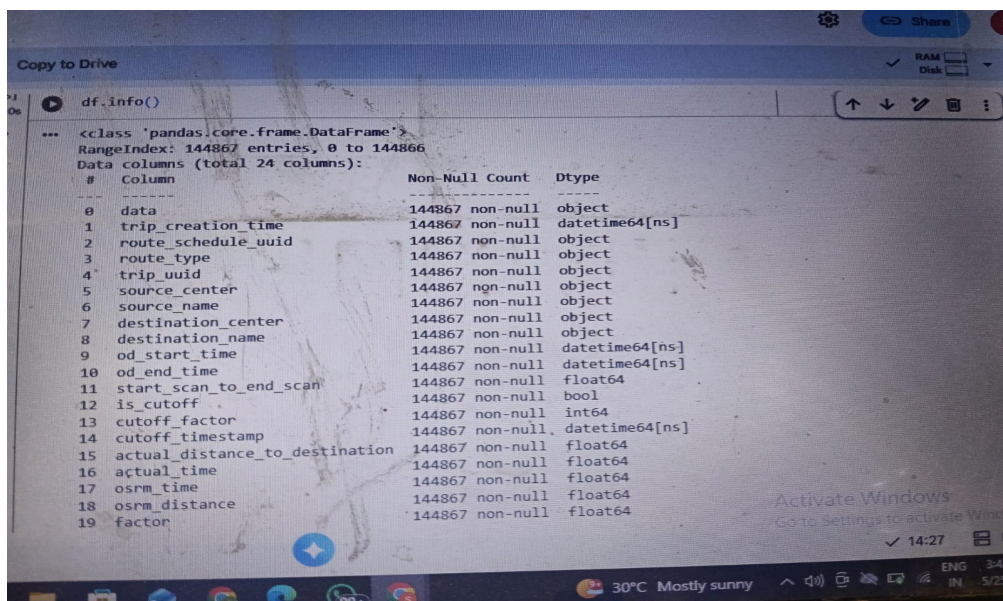
- **Optimize Segment Handoffs:** Investigate and optimize processes between trip segments to account for unrecorded times and improve data integrity for segment times.
- **Further Steps for ML:** Continue with feature selection, model training, validation, and hyperparameter tuning.

```
datetime_cols = ['trip_creation_time', 'od_start_time',
                 'od_end_time', 'cutoff_timestamp']
for col in datetime_cols:
    df[col] = pd.to_datetime(df[col], format='mixed')

df['source_name'].fillna('Unknown', inplace=True)
df['destination_name'].fillna('Unknown', inplace=True)

df.info()
```

Result:



```
df.info()
```

#	Column	Non-Null Count	Dtype
0	data	144867 non-null	object
1	trip_creation_time	144867 non-null	datetime64[ns]
2	route_schedule_uuid	144867 non-null	object
3	route_type	144867 non-null	object
4	trip_uuid	144867 non-null	object
5	source_center	144867 non-null	object
6	source_name	144867 non-null	object
7	destination_center	144867 non-null	object
8	destination_name	144867 non-null	object
9	od_start_time	144867 non-null	datetime64[ns]
10	od_end_time	144867 non-null	datetime64[ns]
11	start_scan_to_end_scan	144867 non-null	float64
12	is_cutoff	144867 non-null	bool
13	cutoff_factor	144867 non-null	int64
14	cutoff_timestamp	144867 non-null	datetime64[ns]
15	actual_distance_to_destination	144867 non-null	float64
16	actual_time	144867 non-null	float64
17	osrm_time	144867 non-null	float64
18	osrm_distance	144867 non-null	float64
19	factor	144867 non-null	float64

```
agg_funcs = {
    'data': 'first',
    'trip_creation_time': 'first',
    'route_schedule_uuid': 'first',
```

```

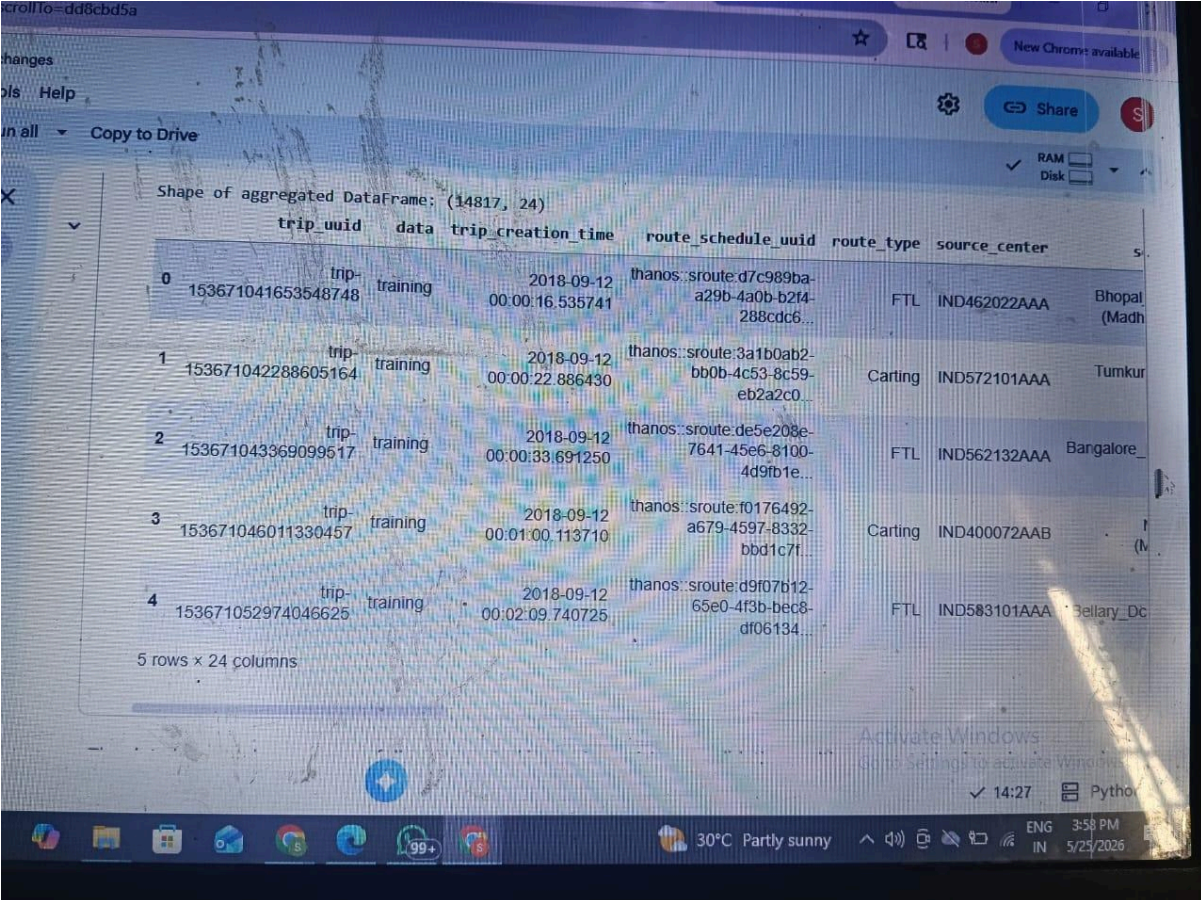
        'route_type': 'first',
        'source_center': 'first',
        'source_name': 'first',
        'destination_center': 'last',
        'destination_name': 'last',
        'od_start_time': 'first',
        'od_end_time': 'last',
        'start_scan_to_end_scan': 'max', # Max duration from start
to end scan of any segment
        'is_cutoff': 'max', # If any segment was cutoff, the trip
was cutoff
        'cutoff_factor': 'mean', # Average cutoff factor
        'cutoff_timestamp': 'last',
        'actual_distance_to_destination': 'last', # Total actual
distance
        'actual_time': 'last', # Total actual time
        'osrm_time': 'last', # Total OSRM time
        'osrm_distance': 'last', # Total OSRM distance
        'factor': 'mean', # Average factor
        'segment_actual_time': 'sum', # Sum of segment actual times
        'segment_osrm_time': 'sum', # Sum of segment OSRM times
        'segment_osrm_distance': 'sum', # Sum of segment OSRM
distances
        'segment_factor': 'mean' # Average segment factor
    }

# Group by 'trip_uuid' and apply aggregation functions
df_agg = df.groupby('trip_uuid').agg(agg_funcs).reset_index()

# Display the shape of the aggregated DataFrame and its head
print("Shape of aggregated DataFrame:", df_agg.shape)
display(df_agg.head())

```


Result:



Shape of aggregated DataFrame: (14817, 24)

	trip_uuid	data	trip_creation_time	route_schedule_uuid	route_type	source_center
0	153671041653548748	trip-training	2018-09-12 00:00:16.535741	thanos:sroute:d7c989ba-a29b-4a0b-b214-288cdc6...	FTL	IND462022AAA Bhopal (Madh
1	153671042288605164	trip-training	2018-09-12 00:00:22.886430	thanos:sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	IND572101AAA Tumkur
2	153671043369099517	trip-training	2018-09-12 00:00:33.691250	thanos:sroute:de5e208e-7641-45e6-8100-4d9fb1e...	FTL	IND562132AAA Bangalore
3	153671046011330457	trip-training	2018-09-12 00:01:00.113710	thanos:sroute:f0176492-a679-4597-8332-bbd1c7f...	Carting	IND400072AAB (M
4	153671052974046625	trip-training	2018-09-12 00:02:09.740725	thanos:sroute:d9f07b12-65e0-4f3b-bec8-df06134...	FTL	IND583101AAA Bellary_Dc

5 rows x 24 columns

```
# 1. Extract features from Trip_creation_time
df_agg['trip_creation_month'] =
df_agg['trip_creation_time'].dt.month
df_agg['trip_creation_year'] =
df_agg['trip_creation_time'].dt.year
df_agg['trip_creation_day'] = df_agg['trip_creation_time'].dt.day
df_agg['trip_creation_day_of_week'] =
df_agg['trip_creation_time'].dt.dayofweek
df_agg['trip_creation_hour'] =
df_agg['trip_creation_time'].dt.hour
```

```
# 2. Calculate the time taken between od_start_time and
od_end_time
df_agg['trip_duration_minutes'] = (df_agg['od_end_time'] -
df_agg['od_start_time']).dt.total_seconds() / 60
```

```
# 3. Extract features from Destination Name and Source Name
# Define a function to extract City and State
```

```

def extract_city_state(name):
    if pd.isna(name) or name == 'Unknown':
        return 'Unknown', 'Unknown'
    parts = name.split('_')
    city = parts[0] if len(parts) > 0 else 'Unknown'
    state = 'Unknown'
    if '(' in name and ')' in name:
        state = name.split('(')[-1].replace(')', '').strip()
    return city, state

df_agg[['source_city', 'source_state']] =
df_agg['source_name'].apply(lambda x:
pd.Series(extract_city_state(x)))
df_agg[['destination_city', 'destination_state']] =
df_agg['destination_name'].apply(lambda x:
pd.Series(extract_city_state(x)))

df_agg['time_diff'] = df_agg['actual_time'] -
df_agg['start_scan_to_end_scan']

df_agg['distance_diff'] =
df_agg['actual_distance_to_destination'] -
df_agg['osrm_distance']

df_agg['segment_time_diff'] = df_agg['segment_actual_time'] -
df_agg['actual_time']

df_agg['osrm_distance_consistency'] =
df_agg['segment_osrm_distance'] - df_agg['osrm_distance']

display(df_agg[['trip_creation_month', 'trip_creation_year',
'trip_creation_day', 'trip_creation_day_of_week',
'trip_creation_hour',
'trip_duration_minutes', 'source_city',
'source_state', 'destination_city', 'destination_state',
'time_diff', 'distance_diff', 'segment_time_diff',
'osrm_distance_consistency']].head())
df_agg.info()

```


changes
ols Help

in all Copy to Drive

Share

RAM Disk

Shape of aggregated DataFrame: (14817, 24)

	trip_uid	data	trip_creation_time	route_schedule_uid	route_type	source_center	destination_center
0	153671041653548748	training	2018-09-12 00:00:16.535741	thanos:route:d7c989ba-a29b-4a0b-b2f4-288cdc6...	FTL	IND462022AAA	Bhopal (Madh...
1	153671042288605164	training	2018-09-12 00:00:22.886430	thanos:route:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	Carting	IND572101AAA	Tumkur
2	153671043369099517	training	2018-09-12 00:00:33.691250	thanos:route:de5e208e-7641-456e-8100-4d9fb1e...	FTL	IND562132AAA	Bangalore_
3	153671046011330457	training	2018-09-12 00:01:00.113710	thanos:route:f0176492-a679-4597-8332-bbd1c71...	Carting	IND400072AAB	(M...
4	153671052974046625	training	2018-09-12 00:02:09.740725	thanos:route:d9f07b12-65e0-4f3b-bec8-df06134...	FTL	IND583101AAA	Bellary_Dc...

5 rows x 24 columns

Activate Windows
Go to Settings to activate Windows.

14:27 Python

30°C Partly sunny 3:58 PM 5/25/2026

```
df.info()
```

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   data                   144867 non-null object
1   trip_creation_time     144867 non-null datetime64[ns]
2   route_schedule_uid    144867 non-null object
3   route_type            144867 non-null object
4   trip_uid              144867 non-null object
5   source_center         144867 non-null object
6   source_name           144867 non-null object
7   destination_center    144867 non-null object
8   destination_name      144867 non-null object
9   od_start_time         144867 non-null datetime64[ns]
10  od_end_time           144867 non-null datetime64[ns]
11  start_scan_to_end_scan 144867 non-null float64
12  is_cutoff             144867 non-null bool
13  cutoff_factor         144867 non-null int64
14  cutoff_timestamp      144867 non-null datetime64[ns]
15  actual_distance_to_destination 144867 non-null float64
16  actual_time           144867 non-null float64
17  osrm_time             144867 non-null float64
18  osrm_distance         144867 non-null float64
19  factor                144867 non-null float64
```



```

df_agg['trip_creation_month'] =
df_agg['trip_creation_time'].dt.month
df_agg['trip_creation_year'] =
df_agg['trip_creation_time'].dt.year
df_agg['trip_creation_day'] = df_agg['trip_creation_time'].dt.day
df_agg['trip_creation_day_of_week'] =
df_agg['trip_creation_time'].dt.dayofweek
df_agg['trip_creation_hour'] =
df_agg['trip_creation_time'].dt.hour

```

```

df_agg['trip_duration_minutes'] = (df_agg['od_end_time'] -
df_agg['od_start_time']).dt.total_seconds() / 60

```

```

def extract_city_state(name):
    if pd.isna(name) or name == 'Unknown':
        return 'Unknown', 'Unknown'
    parts = name.split('_')
    city = parts[0] if len(parts) > 0 else 'Unknown'
    state = 'Unknown'
    if '(' in name and ')' in name:
        state = name.split('(')[-1].replace(')', '').strip()
    return city, state

```

```

df_agg[['source_city', 'source_state']] =
df_agg['source_name'].apply(lambda x:
pd.Series(extract_city_state(x)))
df_agg[['destination_city', 'destination_state']] =
df_agg['destination_name'].apply(lambda x:
pd.Series(extract_city_state(x)))

```

```

df_agg['time_diff'] = df_agg['actual_time'] -
df_agg['start_scan_to_end_scan']

```

```

df_agg['distance_diff'] =
df_agg['actual_distance_to_destination'] -
df_agg['osrm_distance']

```

```
df_agg['segment_time_diff'] = df_agg['segment_actual_time'] -  
df_agg['actual_time']
```

```
df_agg['osrm_distance_consistency'] =  
df_agg['segment_osrm_distance'] - df_agg['osrm_distance']
```

```
print("Columns in df_agg after feature extraction:")  
for col in df_agg.columns:  
    print(col)
```

```
print(f"\nShape of df_agg after feature extraction:  
{df_agg.shape}")
```


Result:

```
▶ print(f"\nShape of df_agg after feature extraction:")  
... Columns in df_agg after feature extraction:  
trip_uuid  
data  
trip_creation_time  
route_schedule_uuid  
route_type  
source_center  
source_name  
destination_center  
destination_name  
od_start_time  
od_end_time  
start_scan_to_end_scan  
is_cutoff  
cutoff_factor  
cutoff_timestamp  
actual_distance_to_destination  
actual_time  
osrm_time  
osrm_distance  
factor  
segment_actual_time  
segment_osrm_time  
segment_osrm_distance  
segment_factor
```

```

from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

t_stat_actual_osrm, p_value_actual_osrm =
stats.ttest_ind(df_agg['actual_time'], df_agg['osrm_time'])
print(f"T-statistic (actual_time vs osrm_time):
{t_stat_actual_osrm:.2f}")
print(f"P-value (actual_time vs osrm_time):
{p_value_actual_osrm:.4f}")

if p_value_actual_osrm < 0.05:
    print("Reject Null Hypothesis: There is a significant
difference between actual_time and osrm_time.")
else:
    print("Fail to Reject Null Hypothesis: There is no
significant difference between actual_time and osrm_time.")

# Visual analysis: Distribution of actual_time vs osrm_time
plt.figure(figsize=(10, 6))
sns.histplot(df_agg['actual_time'], color='blue', label='Actual
Time', kde=True, stat='density', linewidth=0)
sns.histplot(df_agg['osrm_time'], color='red', label='OSRM Time',
kde=True, stat='density', linewidth=0)
plt.title('Distribution of Actual Time vs OSRM Time')
plt.xlabel('Time (minutes)')
plt.ylabel('Density')
plt.legend()
plt.show()

t_stat_actual_segment, p_value_actual_segment =
stats.ttest_ind(df_agg['actual_time'],
df_agg['segment_actual_time'])
print(f"\nT-statistic (actual_time vs segment_actual_time):
{t_stat_actual_segment:.2f}")
print(f"P-value (actual_time vs segment_actual_time):
{p_value_actual_segment:.4f}")

if p_value_actual_segment < 0.05:

```

```

    print("Reject Null Hypothesis: There is a significant
difference between actual_time and sum of segment_actual_time.")
else:
    print("Fail to Reject Null Hypothesis: There is no
significant difference.")

```

```

plt.figure(figsize=(10, 6))
sns.histplot(df_agg['actual_time'], color='blue', label='Actual
Time', kde=True, stat='density', linewidth=0)
sns.histplot(df_agg['segment_actual_time'], color='green',
label='Segment Actual Time Sum', kde=True, stat='density',
linewidth=0)
plt.title('Distribution of Actual Time vs Sum of Segment Actual
Time')
plt.xlabel('Time (minutes)')
plt.ylabel('Density')
plt.legend()
plt.show()

```

```

contingency_table = pd.crosstab(df_agg['route_type'],
df_agg['is_cutoff'])
chi2_stat, p_value_chi2, dof, expected =
stats.chi2_contingency(contingency_table)

```

```

print(f"\nChi-squared statistic (route_type vs is_cutoff):
{chi2_stat:.2f}")
print(f"P-value (route_type vs is_cutoff): {p_value_chi2:.4f}")

```

```

if p_value_chi2 < 0.05:
    print("Reject Null Hypothesis: There is a significant
association between route_type and is_cutoff.")
else:
    print("Fail to Reject Null Hypothesis: There is no
significant association between route_type and is_cutoff.")

```

```

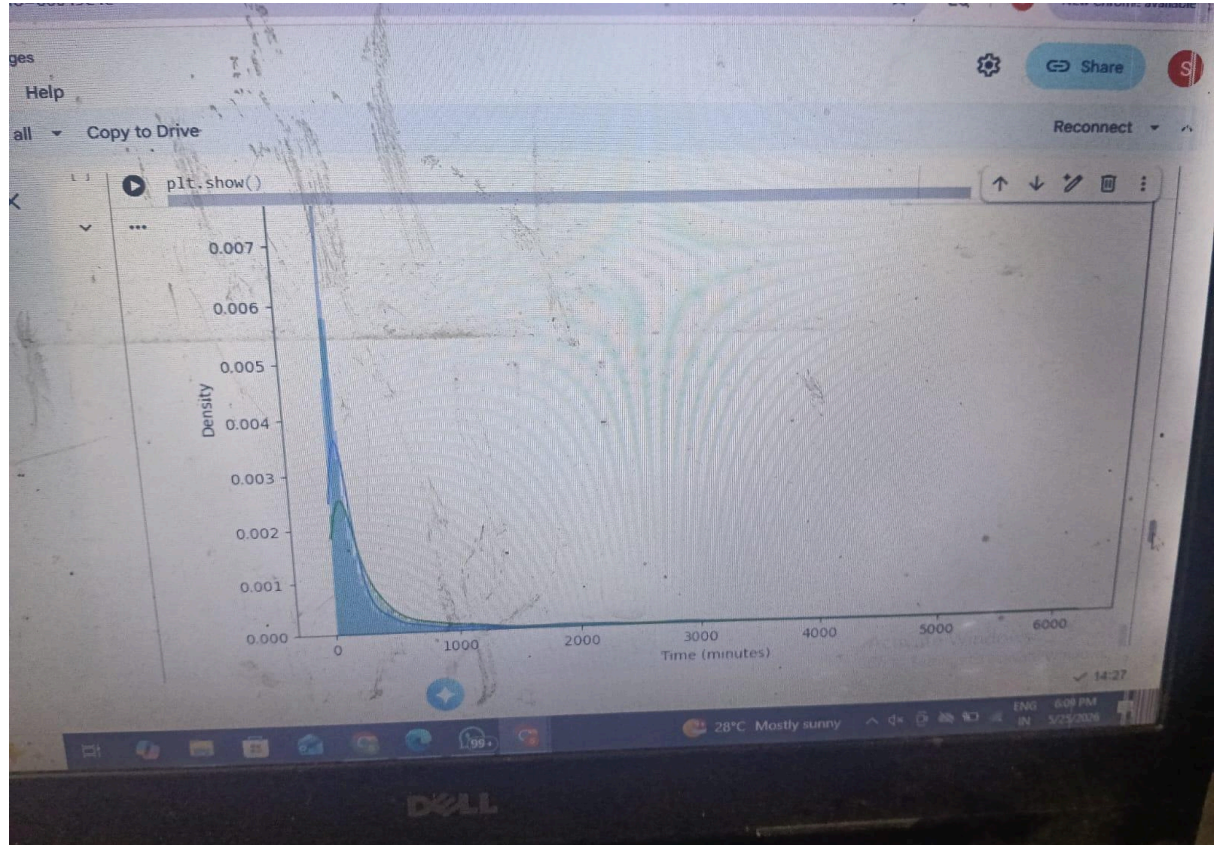
plt.figure(figsize=(8, 5))
sns.countplot(data=df_agg, x='route_type', hue='is_cutoff')
plt.title('Trip Cutoff Status by Route Type')
plt.xlabel('Route Type')

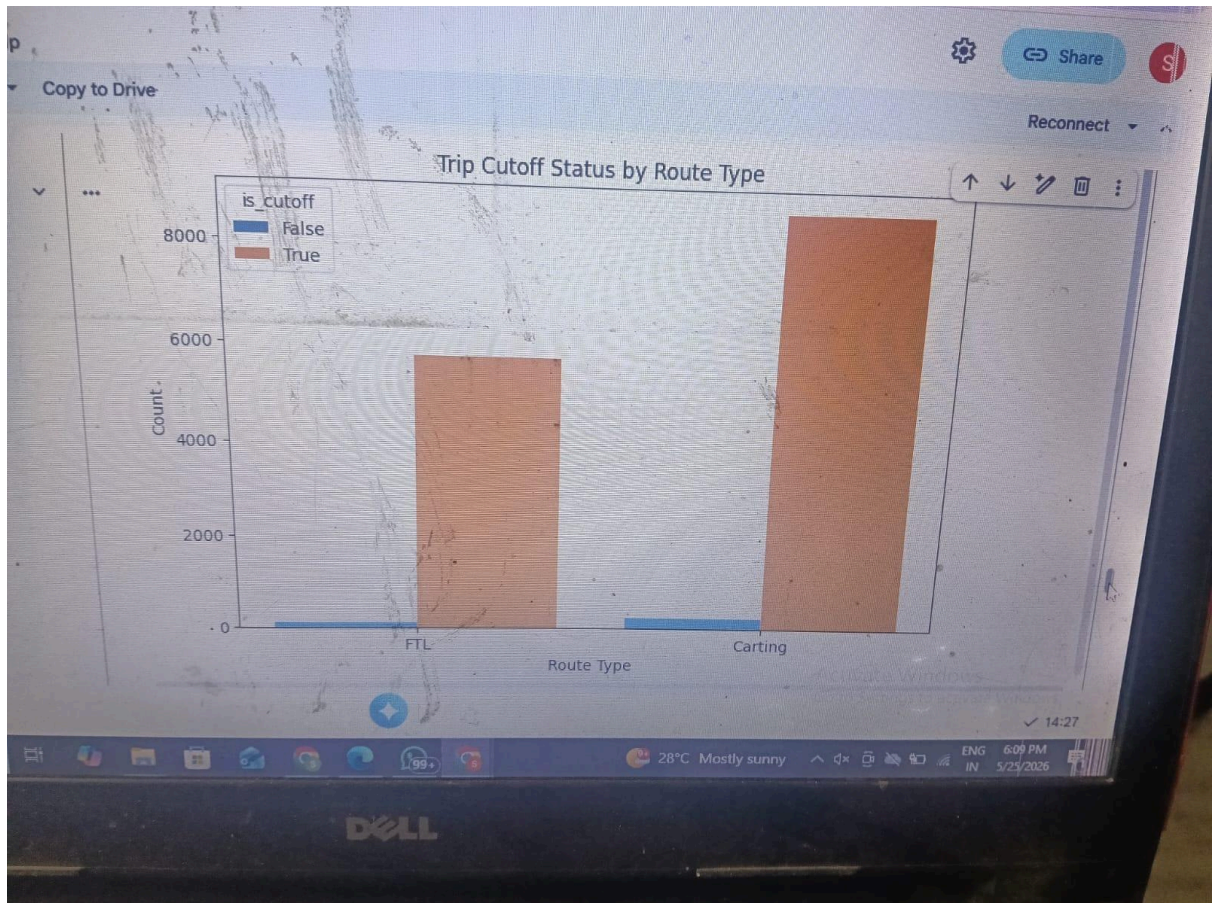
```



```
plt.ylabel('Count')  
plt.show()
```

Result:





```
numerical_cols = df_agg.select_dtypes(include=['float64',
'int64', 'int32']).columns.tolist()

print(f"Numerical columns identified for outlier treatment:
{numerical_cols}")

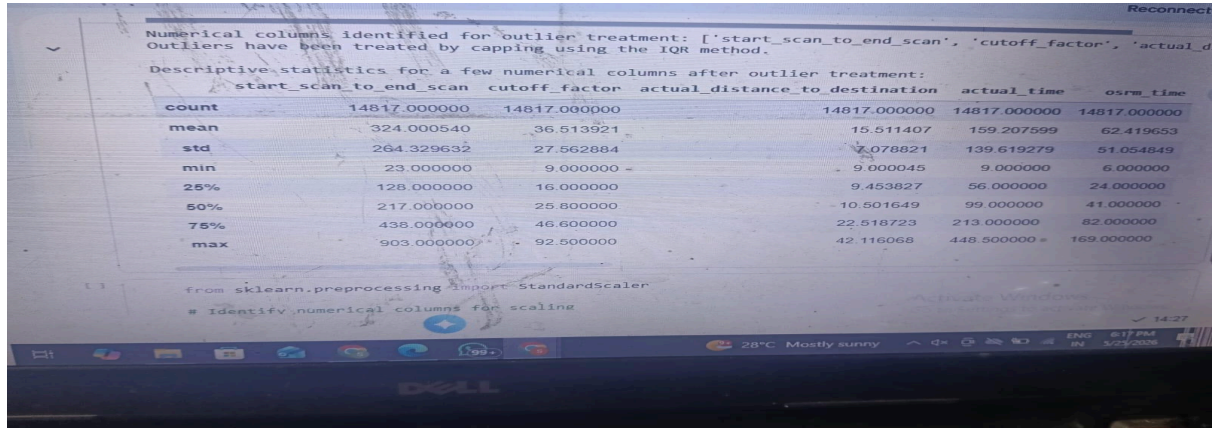
for col in numerical_cols:
    Q1 = df_agg[col].quantile(0.25)
    Q3 = df_agg[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df_agg[col] = df_agg[col].clip(lower=lower_bound,
upper=upper_bound)

print("Outliers have been treated by capping using the IQR
method.")
```

```
print("\nDescriptive statistics for a few numerical columns after
outlier treatment:")
display(df_agg[numerical_cols[:5]].describe())
```

Result:



Numerical columns identified for outlier treatment: ['start_scan_to_end_scan', 'cutoff_factor', 'actual_d']
Outliers have been treated by capping using the IQR method.

Descriptive statistics for a few numerical columns after outlier treatment:

	start_scan_to_end_scan	cutoff_factor	actual_distance_to_destination	actual_time	osrm_time
count	14817.000000	14817.000000	14817.000000	14817.000000	14817.000000
mean	324.000540	36.513921	15.511407	159.207599	62.419653
std	264.329632	27.562884	7.078821	139.619279	51.054849
min	23.000000	9.000000	9.000045	9.000000	6.000000
25%	128.000000	16.000000	9.453827	56.000000	24.000000
50%	217.000000	25.800000	10.501649	99.000000	41.000000
75%	438.000000	46.600000	22.518723	213.000000	82.000000
max	903.000000	92.500000	42.116068	448.500000	169.000000

from sklearn.preprocessing import StandardScaler
Identify numerical columns for scaling

```
from sklearn.preprocessing import StandardScaler
numerical_cols_to_scale =
df_agg.select_dtypes(include=['float64', 'int64',
'int32']).columns.tolist()
```

```
print(f"Numerical columns identified for standardization:
{numerical_cols_to_scale}")
```

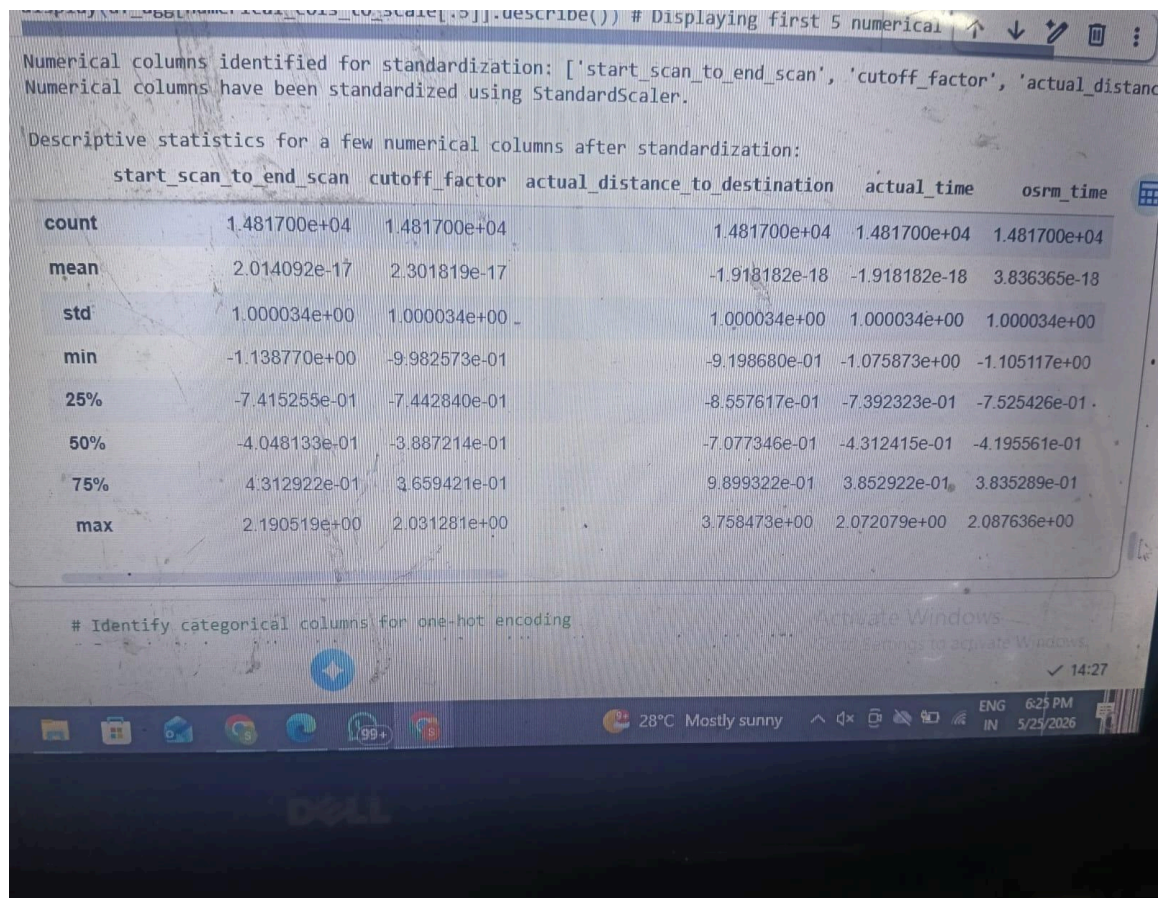
```
scaler = StandardScaler()
```

```
df_agg[numerical_cols_to_scale] =
scaler.fit_transform(df_agg[numerical_cols_to_scale])
```

```
print("Numerical columns have been standardized using
StandardScaler.")
```

```
print("\nDescriptive statistics for a few numerical columns after
standardization:")
display(df_agg[numerical_cols_to_scale[:5]].describe())
```

Result:



```

categorical_cols_encode =
df_agg.select_dtypes(include='object').columns.tolist()

if 'trip_uuid' in categorical_cols_encode:
    categorical_cols_encode.remove('trip_uuid')
if 'route_schedule_uuid' in categorical_cols_encode:
    categorical_cols_encode.remove('route_schedule_uuid')

print(f"Categorical columns identified for one-hot encoding:
{categorical_cols_encode}")

df_agg_encoded = pd.get_dummies(df_agg,
columns=categorical_cols_encode, drop_first=True)

print("Categorical columns have been one-hot encoded.")
print(f"Shape of DataFrame after one-hot encoding:
{df_agg_encoded.shape}")

```

```
display(df_agg_encoded.head())
```

Result:

Copy to Drive

Reconnect

Categorical columns identified for one-hot encoding: ['data', 'route_type', 'source_center']
Categorical columns have been one-hot encoded.
Shape of DataFrame after one-hot encoding: (14817, 3960)

	trip_uuid	trip_creation_time	route_schedule_uuid	od_start_time	od_end_time	start_scan_to
0	trip-153671041653648748	2018-09-12 00:00:16.535741	thanos::route:d7c989ba-a29b-4a0b-b2f4-288cdc6...	2018-09-12 00:00:16.535741	2018-09-13 13:40:23.123744	
1	trip-153671042288605164	2018-09-12 00:00:22.886430	thanos::route:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	2018-09-12 00:00:22.886430	2018-09-12 03:01:59.598855	
2	trip-153671043369099517	2018-09-12 00:00:33.691250	thanos::route:d5e208e-7641-45e6-8100-4d9fb1e...	2018-09-12 00:00:33.691250	2018-09-14 17:34:55.442454	
3	trip-153671046011330457	2018-09-12 00:01:00.113710	thanos::route:f0176492-a679-4597-8332-bbd1c7f...	2018-09-12 00:01:00.113710	2018-09-12 01:41:29.809622	
4	trip-153671052974046625	2018-09-12 00:02:09.740725	thanos::route:d907b12-65e0-4f3b-bec8-df06134...	2018-09-12 00:02:09.740725	2018-09-12 12:00:30.683231	

5 rows x 3960 columns

14:27

28°C Mostly sunny 6:34 PM 8/22/2018

```

display(df_agg['source_state'].value_counts().head(10))

print("\nTop Destination States by number of trips:")
display(df_agg['destination_state'].value_counts().head(10))

plt.figure(figsize=(12, 6))
sns.countplot(y='source_state', data=df_agg,

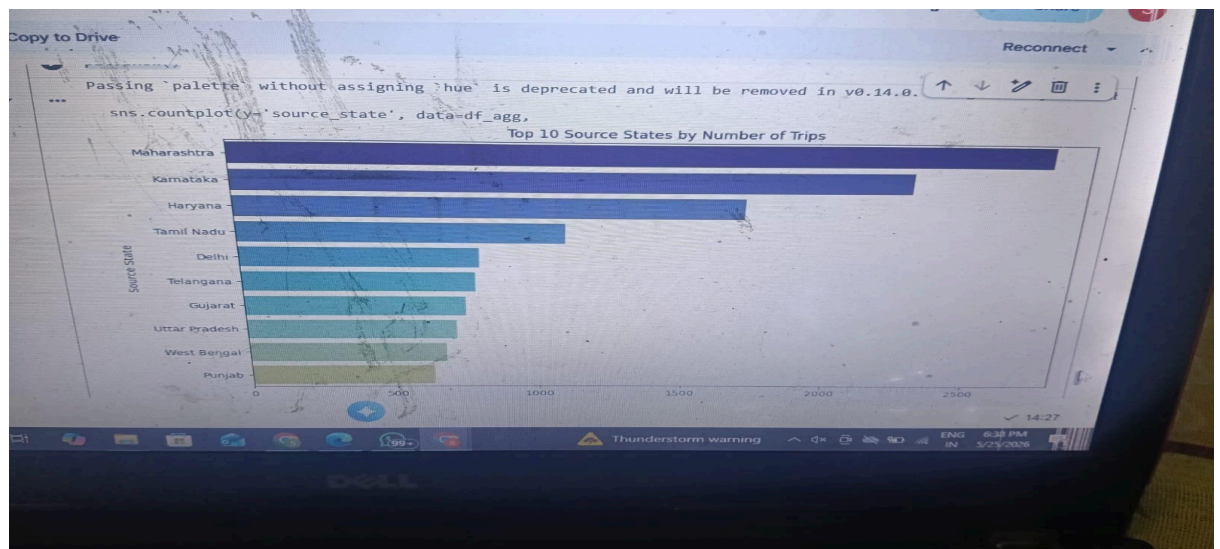
order=df_agg['source_state'].value_counts().index[:10],
            palette='viridis')
plt.title('Top 10 Source States by Number of Trips')
plt.xlabel('Number of Trips')
plt.ylabel('Source State')
plt.show()

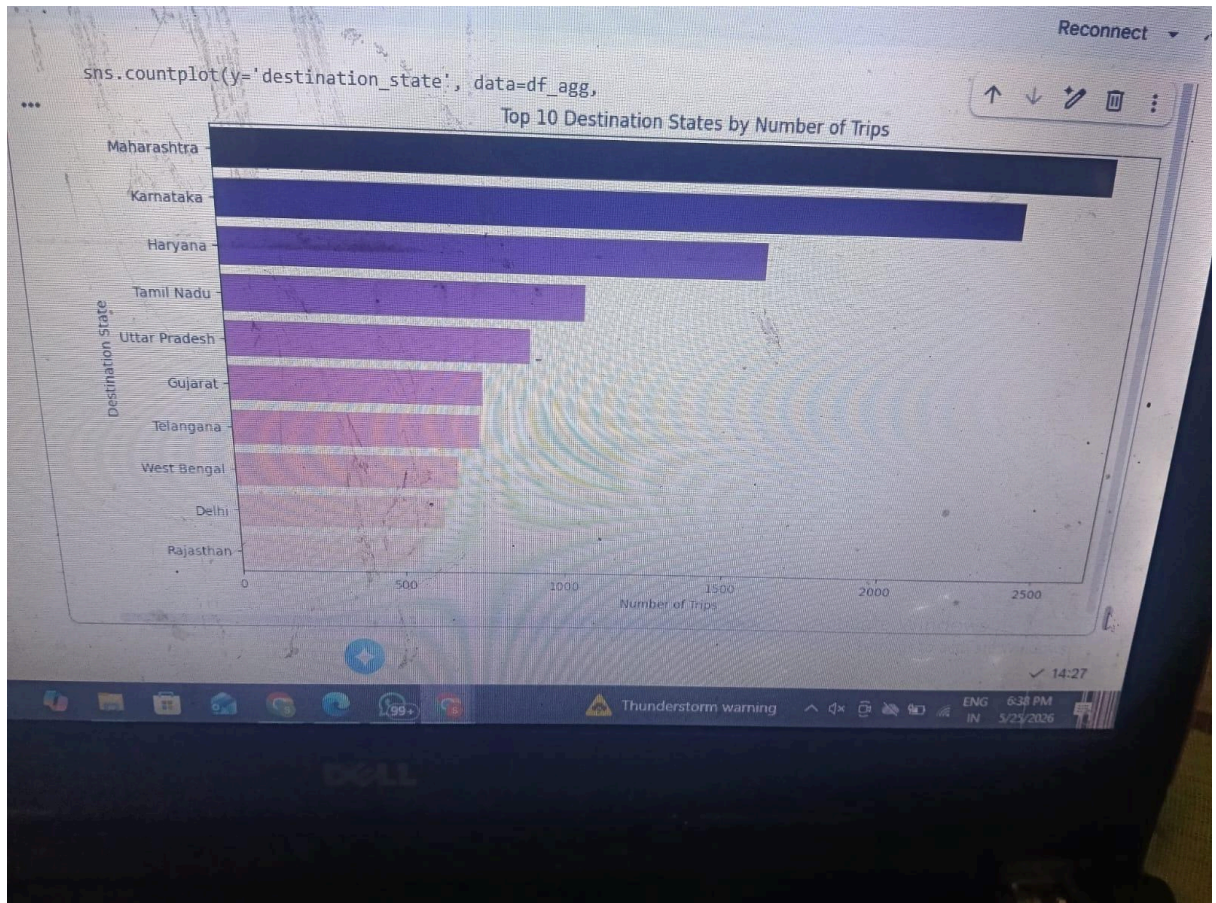
plt.figure(figsize=(12, 6))
sns.countplot(y='destination_state', data=df_agg,

order=df_agg['destination_state'].value_counts().index[:10],
            palette='magma')
plt.title('Top 10 Destination States by Number of Trips')
plt.xlabel('Number of Trips')
plt.ylabel('Destination State')
plt.show()

```

Result:





Sights:

- High operational activity is concentrated in these regions.
- These states are critical for supply chain operations.

Recommendation:

- Improve infrastructure in top source states
- Build regional hubs in high-demand destination states

